

N8N PIPELINES, ENGINEERED

# N8N: No-code today, code you can ship tomorrow.

Orchestration that scales with you.

**+117k**  
downloads

**<50 MB**  
RAM, total

**SQLite**  
WAL state

## Where it hurts today

Three things quietly cost teams time and trust every single week.

READMEs stay  
empty

JSON exports  
un-diffable

Debugging is a  
black box



**wpipe turns these into code you control.**

## What you gain



### Code-first

Pipelines are Python —  
versionable, testable,  
reviewable.



### Resume, don't restart

Every step checkpoints  
to SQLite WAL.



### Featherweight

<50 MB RAM. No broker,  
no daemon.

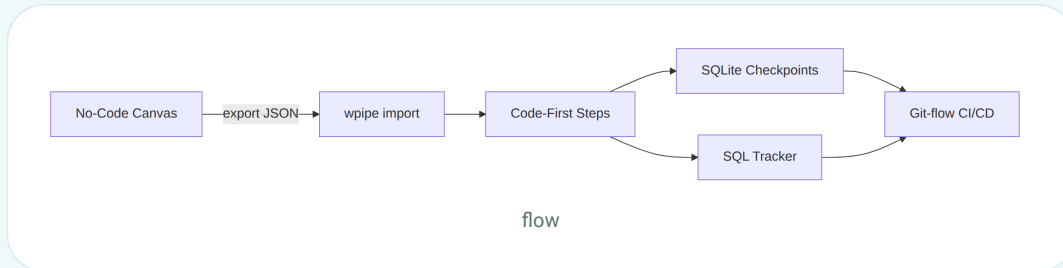
**Crash at step 7? Resume from step 4.**



Every step persists its state to SQLite WAL — a rerun cost you never pay again.

## One pipeline, fully instrumented

State, retries, parallelism and audit — all inside the same run.



wpipe vs. the status quo

| Feature          | WPipe              | Airflow          | n8n       | Celery         | Prefect   | Zapier/Make   |
|------------------|--------------------|------------------|-----------|----------------|-----------|---------------|
| Memory Footprint | < 50MB             | > 2GB            | > 500MB   | > 200MB        | > 500MB   | Cloud / High  |
| Configuration    | Pure Python        | Python/YAML      | Visual UI | Python/Broker  | Python    | Visual UI     |
| Resilience       | SQLite Checkpoints | Postgres/DB      | Database  | Redis/RabbitMQ | Cloud/DB  | None (Manual) |
| Setup Time       | < 1 min            | Hours            | Minutes   | Hours          | Minutes   | Minutes       |
| Cost             | Free/OSS           | OSS (High Infra) | OSS/Paid  | OSS (Infra)    | OSS/Cloud | Per Execution |
| Learning Curve   | Low (Pythonic)     | High             | Medium    | High           | Medium    | Low           |

Based on what the engineering community hits daily.

Answers four questions in one run ↘

Did it run, how far, what failed, and can it resume? Yes, yes, exactly, and automatically.

```
python my_pipeline.py
```

SEE IT IN ACTION

## Six lines. That's the learning curve.

```
from wpipe import Pipeline, step

@step(name="ingest", retry_count=3)
def ingest(data):
    return {"rows": data["payload"]}

pipe = Pipeline(pipeline_name="from_n8n")
pipe.set_steps([ingest])
pipe.run(task_ctx)

CHECKPOINTS · RETRIES · TIMEOUT · PARALLEL · TRACKER
```



### Code-first

Pipelines are Python — versionable, testable, reviewable.



### Resume, don't restart

Every step checkpoints to SQLite WAL.



### Featherweight

<50 MB RAM. No broker, no daemon.

## Try wpipe in your next pipeline

Install the library, wrap your step, and watch checkpoints and docs appear by themselves.

```
pip install wpipe
```